Computational Thinking in Problem Solving

Prepared by: Ryan Austin Fernandez
CSALGCM / ALGOCOM

Agenda

1. Time Complexity Discussion
2. Master’s Theorem Exercises
3. Problem Anatomy
4. Computational Thinking
5. Programming Language Toolkit
6. Fool Arcana Problem Set Solution Sketches

Anatomy of a Problem (on most online judges)

1. Title and Time/Space Limits
2. Context/Story
3. Input Format
4. Output Format
5. Constraints
6. Sample Input
7. Sample Output

Title and Time/Space Limits

- Title is the name of the problem.
- Problems are usually also denoted by a letter.
- Time limit is how much time your program has to output its answer.
- Space limit is how much memory your program can use.

Context/Story

- What's the problem?
- Strip the narrative.
- Sometimes full of worthless details
- There should be no ambiguity.

John was given X pesos by his older sister Kate. He wants to go to the market to buy some candies. However, his sister told him to get her K apples. There are N stores Si numbered 1 to N. The price of an apple and a candy at store Si is Ai and Ci respectively. ***John does not need to buy the apples and candies at the same store. John always has enough money to buy K apples at one of the stores.*** *What is the maximum number of candies John can buy?*

Input/Output Format

- Precise formatting of I/O
- There is no room for error.
- Problemsetters use input/output validator programs to ensure each one strictly adheres to the format. If an I/O file fails the validation, it fails the test case.

Input/Output Format

Input begins with a line containing a number T, denoting the test cases. Each test case begins with a line containing a real number X and two integers, N and K, each separated by a single space. The ith line of the next N lines each contain two non-negative real numbers Ai and Ci also separated by a single space. There will be no leading or trailing spaces.

For each test case, output a single line containing an integer which is the maximum number of candies John can buy.

Constraints

- This gives you the range of values for the inputs (and sometimes the outputs).
- This is very important in selecting an algorithm.
  - Small constraints allow you to use slower algorithms
  - Larger constraints require more complex approaches. Sometimes, a formula has to be derived which can be computed in constant time.

1 <= T <= 10

0 <= X, K, Ai, Ci <= 10^9

1 <= N <= 10^5

Sample Input/Output

- Problemsetters give these to help you confirm your understanding of the problem.
- These test cases will not be exhaustive. They will usually not include the tricky cases. That’s your job to figure out.

Sample Input/Output

Sample Input

```
2
20.0 2 3
1 5
2 1
100.0 2 10
11 1
10 1
```

Sample Output

```
17
0
```

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Accepted - Your code passed all the test cases!

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Accepted - Your code passed all the test cases!
- Wrong Answer - Your code failed one of the test cases. :(

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Accepted - Your code passed all the test cases!
- Wrong Answer - Your code failed one of the test cases. :(
- Time Limit Exceeded - Your program did not terminate in time.

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Accepted - Your code passed all the test cases!
- Wrong Answer - Your code failed one of the test cases. :(
- Time Limit Exceeded - Your program did not terminate in time.
- Runtime Error - Your program did not terminate properly or without errors.

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Memory Limit Exceeded - Your program used too much RAM.

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Memory Limit Exceeded - Your program used too much RAM.
- Output Limit Exceeded - Your program produced too much output for the checker to handle.

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Memory Limit Exceeded - Your program used too much RAM.
- Output Limit Exceeded - Your program produced too much output for the checker to handle.
- Presentation Error - Your output is correct but does not match the output format exactly.

After submitting a solution… what happens next?

You receive one of the following verdicts:

- Memory Limit Exceeded - Your program used too much RAM.
- Output Limit Exceeded - Your program produced too much output for the checker to handle.
- Presentation Error - Your output is correct but does not match the output format exactly.
- Compile Error - Your code did not compile properly.

Computational Thinking in Problem Solving

What is **“computational thinking”?**

Computational Thinking in Problem Solving

What is **“computational thinking”?**

There is no *definite* consensus.

Computational Thinking in Problem Solving

What is **“computational thinking”?**

There is no *definite* consensus.

Computational thinking is a way of approaching problem solving using the thought processes normally used by computer scientists.

CT Concepts

- Problem Decomposition
  - Taking a complex problem and breaking it down into simpler subproblems
- Abstraction
  - Ignoring information that is unnecessary to solve a problem
- Algorithmic Thinking
  - Providing a finite sequence of unambiguous instructions to solve a problem.
- Evaluation
  - Correctness
  - Efficiency
  - Elegance
  - Usability

Programming Language Toolkit

When planning to code algorithmic solutions and logic in a new language, you **must know** the following features of your selected language.

- I/O (File or Standard I/O)
- Sorting
- Arrays/Lists
- Stacks/Queues/Priority Queues
- Binary Search Trees
- Hash Maps

Fool Arcana Problem Set Solutions

Hello World

Trivial problem. Just print “Hello World!”

Solving For Carrots

Also trivial. Once you understand the problem, you just have to print P.

Judging Moose

- Check if both 0 first
- Otherwise, check if l = r (Even)
- Otherwise, handle odd case

Quick Brown Fox

- Maintain a boolean array with an entry for each letter.
- Track which letters appear.
- Print missing letters if not pangram

Pig Latin

- Tricky aspect: **y** is a vowel for the purpose of this problem.
- Otherwise, straightforward implementation problem.

Boss Battle

- Try to find the pattern for lower **n**.
- Formula is just **n - 2**.
- Tricky case: n = 1 or n = 2 (answer is 1)

Dyslectionary

- Input and output format is trickiest part. This problem tests your knowledge of how to use your language's I/O.
- Reverse all words (most languages have a string reverse functionality).
- Sort (use libraries)
- Reverse the words again
- Track maximum word length
- Output the answer with proper spacing and word width.

Game of Throwns

- Use a stack to track latest passes.
- Pop from stack during “undo” operations.
- Use modulo arithmetic to manage the circle.
- For negative passes, use formula (a - b) mod m = (a - b + m) mod m

Factstone Benchmark

- This is your intro to *hardware limitations***.**
- Naive approach is to compute factorial for any test case.
- However, this will lead to **integer overflow**.
- Instead, realize that the number of bits in any number is proportional to its logarithm (base 2). Specifically, num_bits = floor(original_number) + 1.
- Additional insight: log (ab) = log a + log b
- Get the logarithm of the factorial and use the formula above.

Sjecista

- This requires some patience.
- The answer is a simple pattern, with a caveat: **the pattern is a degree 4 polynomial.**
- This then requires you to solve the answer for 3 <= N <= 7 and realize that it's a polynomial pattern.
- Answer = $\frac{n}{4}\left(\frac{(n-2)^2(n-3)}{2} - \frac{(n-3)(n-2)(2n-5)}{6}\right)$

Thank You!